ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: 06-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: **https://github.com/192524173simats-karthikreddy/DSA01-192524173.git**

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time

complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

- The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each
- How the same problem would be written in a procedural style, and what changes in an object-oriented style
- The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system
- The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)

You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks

Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one place only.

Discuss:

- The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples
- #define, const and constexpr constants, and why constexpr is preferred for the fare table
- enum and enum class for zones, and why enum class is safer
- Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

- The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system

- Why money should be stored in integer paise and not as double rupees
- Type compatibility rules, and what the compiler does with int, double and bool values in the same expression
- The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

- Integer division and integer-to-double conversion
- Comparison of signed and unsigned values, and the compiler warning it produces
- Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them •

  static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


int main()

{

   int a = 7, b = 2;

double c = a / b;
```

```
double d =
static_cast<double>(a) /
b;    std::cout << c << ' '
<< d << '\n';

   char ch = 'A' + 2;    std::cout << ch <<
' ' << ch + 1 << '\n';

   bool flag = 5;    std::cout << flag << ' ' <<
std::boolalpha << flag << '\n';

   unsigned int u = 1;
   int s = -1;    std::cout << (s < u ? "s < u" : "s
>= u") << '\n';
}
```

My predicted output (write before running):

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom’s Level: K5 – Evaluate

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

- Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment
- The difference between x++ and ++x, and why i = i++ is a mistake
- The result of / and % with negative operands
- Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


int main()

{    int x = 5, y = 3;    std::cout << x + y * 2 << ' '

<< (x + y) * 2 << '\n';


  int z = x++ + ++y;    std::cout << z << ' '

<< x << ' ' << y << '\n';


  std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';    std::cout << 10 / 3

<< ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';


  int k = 0;    for (int i = 0; i < 10; i

+= 3) k += i;    std::cout << k <<

'\n';

}
```

My predicted output (write before running):

For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's journeys.

Discuss:

- When to use if–else, a switch on the zone, and the conditional operator
- How a for loop, a range-based for loop and a while loop could process the journeys, and which is best
- Where break and continue are needed (for example, skipping an invalid journey) •

  What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream>


enum class Zone { Inner, Middle, Outer };


constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
```

```
constexpr int PEAK_SURCHARGE_PERCENT = 20; constexpr long long
MIN_BALANCE_PAISE = 5000;     // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
   // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add
   //   PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only

}

bool canTravel(long long balance, long long f)
{
   // TODO 2: Return true only if the balance left after paying the fare is at least
   //   MIN_BALANCE_PAISE

}

int main()
{
   long long balance = 10000;                    // Rs 100.00 in paise
struct Trip { Zone zone; bool peak; };
   Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
for (const Trip& t : trips) {        long long f = fare(t.zone, t.peak);        if
(canTravel(balance, f)) {            balance -= f;
          std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                    << std::setfill('0') << f % 100 << std::setfill(' ');
       } else {
          std::cout << "Denied (low balance)";
       }
       std::cout << "  balance Rs " << balance / 100 << '\n';
   }
```

}

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate Rewrite

the fare logic in two styles.

Discuss:

- A procedural version with separate variables and free functions
- An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()
- How the class prevents a negative balance
- Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

- Reading values with cin and checking cin.fail() when the operator types letters instead of numbers
- Clearing the error state with cin.clear() and discarding the bad input
- Formatting money with <iomanip> (setw, setfill, fixed, setprecision)
- Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

- Normal journeys in each zone, peak and off-peak
- Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50
- Invalid cases: an unknown zone, a negative recharge, letters typed for the hour • A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

- Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ
- Whether int overflows for the network total
- The compiler warnings you received with -Wall -Wextra and what each meant • Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

- The types, constants and classes you recommend and why
- How a fare revision will be carried out
- The limitations of your design
- Whether the design remains correct for the whole network

Your recommendation must be based on evidence from your own experiments.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Object-Oriented Analysis	K5 – Evaluate	4

Task 2 – Tokens and Constants	K5 – Evaluate	3
Task 3 – Data Types and Compatibility	K5 – Evaluate	4
Task 4 – Implicit Conversions	K5 – Evaluate	4
Task 5 – Operators and Precedence	K5 – Evaluate	4
Task 6 – Control Structures	K5 – Evaluate	4
Task 7 – Procedural vs Object-Oriented	K5 – Evaluate	4
Task 8 – Input and Output	K5 – Evaluate	4
Task 9 – Implementation & Testing	K5 – Evaluate	4
Task 10 – Expectation vs Execution	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		42
Task 12 – Day-4 Design Challenge	K5 – Evaluate	8
TOTAL		50

Submission Requirements

- GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TASK 1 - Understand the Problem and Identify the Objects

1.1 Objects in the problem, with their data and behaviour

An object is justified only if it owns data that must stay consistent and behaviour that operates on that data. The table lists each candidate object and why it deserves to be one.

Object	Data (private state)	Behaviour	Why it is an object
SmartCard	`balance_` (long long, paise)	`canTravel(fare)`, `deduct(fare)`, `recharge(p)`, `balance()`	Owns the one value that must never go wrong. All rules about the balance live with it.
Passenger	name, category (General / Student / Senior)	`isConcession()`	Decides whether the concession applies. Keeps person data away from money data.
Journey	zone (farthest zone), hour, fare computed	`process(card)`	One tap at the gate: asks for the fare, checks the card, deducts. It is a short-lived record of one event.
Zone	Inner, Middle, Outer (enum class)	none (a closed set of values)	Prevents illegal zone numbers and stops zones being mixed with plain integers.
FareTable	base fares per zone, peak percent, peak hours	`fare(zone, hour)`, `isPeak(hour)`	Holds every fare constant in one place so a revision is made once.

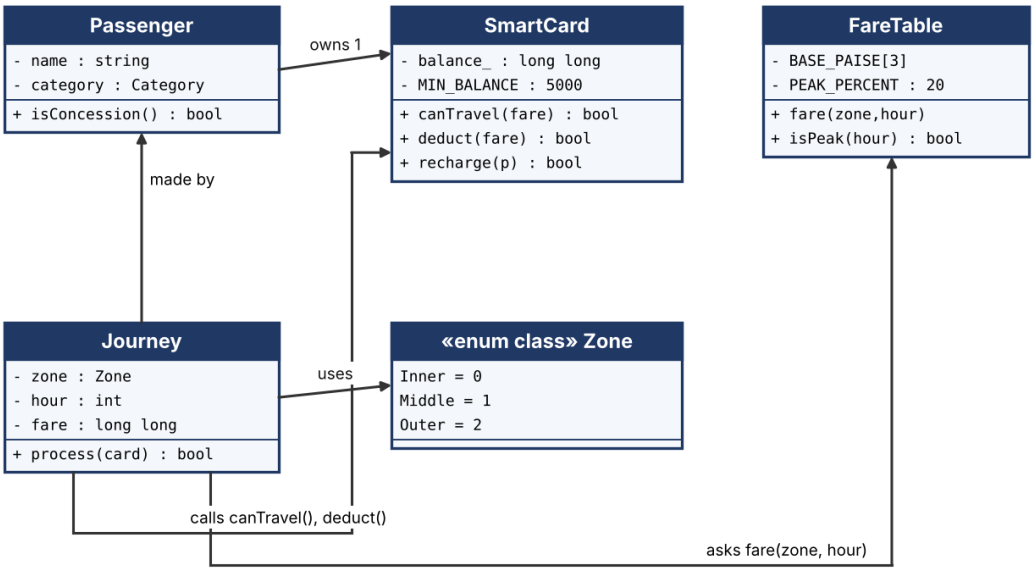


Figure 1.1 - UML class diagram of the original design

1.2 Procedural style versus object-oriented style

In a **procedural** program the data and the functions are separate. The balance would be a global variable and `fare()`, `canTravel()` and `deduct()` would be free functions that any part of the program can call. This is how the first version of the gate software was written, and it explains the reported faults: the fare values were typed in several functions, the peak check was copied with slightly different comparisons, and any function could write a negative value into the balance.

In the **object-oriented** style the balance becomes a private member of `SmartCard`. The only ways to change it are `deduct()` and `recharge()`, and both check the rules first. The fare values move into one `FareTable`. What changes is not the arithmetic but **who is allowed to touch what**: a rule is written once, next to the data it protects, and the compiler refuses code that bypasses it.

1.3 The four principles of OOP in this system

Principle	Where it appears	Concrete example
Encapsulation	`SmartCard::balance_` is private	A gate cannot write `card.balance_ = -5;` it must call `deduct()`, which refuses an unsafe fare.

Principle	Where it appears	Concrete example
Abstraction	Gate code sees only `canTravel()` and `deduct()`	The gate does not know the Rs 50 minimum or how the balance is stored (paise or otherwise).
Inheritance	Card types and passengers	`StudentCard` and `SeniorCard` could derive from `SmartCard` and reuse everything except the concession rule.
Polymorphism	A virtual `fareFor()` per passenger category	`std::vector<SmartCard*>` can hold different cards, and `card->fareFor(base)` picks the right concession at run time.

Inheritance and polymorphism are not needed for the first version (a single `SmartCard` plus a `bool concession` is enough). They become useful in Task 12 if card types multiply, so they are named here but not forced into the code.

1.4 Rules every correct journey must satisfy

#	Rule	Justification
R1	Balance after paying must be at least 5000 paise (Rs 50).	Stated business rule; tested at the boundary in Task 9.
R2	The balance can never be negative.	A negative balance once appeared; R1 plus a checked recharge make it impossible.
R3	Fare = base fare of the farthest zone: Rs 10 / 20 / 30.	A farthest-zone fare needs one comparison, not a sum over zones.
R4	Peak = hour in [8,10) or [17,20); surcharge = 20%.	Half-open intervals give exactly one answer for 10 AM and 8 PM.
R5	All money is integer paise; percentages use integer arithmetic.	Removes the 'off by a paisa' fault (Task 3 and Task 10).
R6	Concession (student/senior) reduces the fare (assumed 50%).	Rate is not given, so it is a named constant.

#	Rule	Justification
R7	Invalid input (unknown zone, hour outside 0-23, non-positive recharge) is rejected, never guessed.	A gate must fail safe.
R8	Every fare constant is defined once.	A revision must be one edit, not a search.

1.5 Assumptions

- One journey is priced by the **farthest zone** reached; a passenger enters and exits once per journey.
- Hour is an integer 0-23 taken from the gate clock; minutes do not matter because the peak windows start and end on the hour.
- A denied journey changes nothing on the card.
- Scale: 20,000, 500,000 and 3,000,000 journeys per day are processed one at a time, so the work grows linearly with the number of journeys.

Alternative Concept and Complexity Analysis

Alternative: a purely procedural program with a global balance and free functions. Pseudocode:

Pseudocode (procedural)

```
ALGORITHM ProceduralGate
GLOBAL balance <- 10000, BASE <- [1000, 2000, 3000]
FUNCTION fare(zone, hour):
    base <- BASE[zone]
    IF peak(hour) THEN base <- base + base*20/100
    RETURN base
FUNCTION pay(zone, hour):
    f <- fare(zone, hour)
    IF balance - f >= 5000 THEN balance <- balance - f; RETURN
true
    RETURN false
```

Measure	Procedural	Object-oriented	Comment
Time per journey	O(1)	O(1)	One table lookup, a few comparisons, one subtraction in both.
Memory per card	O(1): 8 bytes	O(1): 8 bytes (`balance_`)	No extra field per object (no virtual functions used).
Total memory, c cards	O(c)	O(c)	Identical layout.
Total time, n journeys	O(n)	O(n)	20,000 -> 500,000 -> 3,000,000 journeys scales linearly.
Places edited when one fare changes	Every copy of the number (several in the legacy code)	1 (the `constexpr` table)	The real difference between the styles.
Bypassing a rule	Possible from any function	Rejected by the compiler	Shown in Task 7.

Conclusion: time and memory do not change, so the choice is made on **maintainability and safety**. The object-oriented design is retained for the final system.

TASK 2 - Use Tokens, Keywords, Identifiers and Constants Correctly

2.1 The five kinds of tokens

The compiler first splits the source text into tokens. The table classifies the tokens of this line from the fare function:

Line analysed

```
if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
```

Token kind	Meaning	Examples from the fare program
Keywords	Reserved words with fixed meaning	`if`, `switch`, `case`, `break`, `return`, `constexpr`, `enum`, `class`, `long`, `bool`, `const`
Identifiers	Names chosen by the programmer	`fare`, `canTravel`, `balance_`, `BASE_PAISE`, `Zone`, `peak`
Constants (literals)	Fixed values written in the source	`1000`, `20`, `5000LL`, `true`, `'A'`, `"Fare Rs "`
Operators	Symbols that compute	`+=`, `*`, `/`, `-`, `>=`, `&&`, `||`, `<<`, `?:`
Punctuators	Separators and grouping	`;` , `(` `)` `{` `}` `[` `]` `::`

2.2 #define, const and constexpr

Feature	#define	const	constexpr
Handled by	Preprocessor (text replacement)	Compiler	Compiler
Has a type	No	Yes	Yes
Obeys scope	No (global from the #define line)	Yes	Yes
Value known at compile time	Yes (it is the text)	Not guaranteed	**Guaranteed** (or a compile error)
Usable as array size / case label	Yes	Only if initialised by a constant	Yes

Feature	#define	const	constexpr
Visible in debugger / error messages	No (replaced before compiling)	Yes	Yes
Safe in expressions	No: `x * 2` expansion bugs	Yes	Yes

`constexpr` is preferred for the fare table because the compiler **proves** that the values are compile-time constants, gives them a type (`long long`), restricts them to their scope, and rejects any attempt to change them. A `#define` gives none of these protections.

Source: Task02/task02.cpp

```
// Task 2 - #define vs constexpr vs enum class
#include <iostream>
#define BASE_INNER 1000              // macro: no type, no
scope
#define PEAK_PCT   20
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};   //
typed, scoped, compile-time
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
enum class Zone { Inner, Middle, Outer };   // scoped, no
implicit int conversion
enum OldZone { OInner, OMiddle, OOuter };   // plain enum:
leaks names, converts to int
int main()
```

The single set of constants used by the whole program

Constants (one place only)

```
constexpr long long BASE_PAISE[]            = {1000, 2000,
3000}; // Rs 10, 20, 30
constexpr int       PEAK_SURCHARGE_PERCENT  = 20;
constexpr int       CONCESSION_PERCENT      = 50;
// assumption (not given)
```

```
constexpr long long MIN_BALANCE_PAISE       = 5000;
// Rs 50 must remain
constexpr int       PEAK_START_1 = 8,  PEAK_END_1 = 10;
// [8,10)
constexpr int       PEAK_START_2 = 17, PEAK_END_2 = 20;
// [17,20)
```

2.3 enum and enum class for zones

- A plain `enum` puts `Inner`, `Middle`, `Outer` in the surrounding scope (name clashes) and converts silently to `int`, so `fare(2, ...)` or `zone == 7` compile without complaint.
- An `enum class` keeps names inside `Zone::` and does **not** convert to `int` without a `static_cast`. A wrong value such as a bare integer is a compile-time error, which is exactly the safety a gate needs.
- The program output below demonstrates the difference (the plain enum became `int` with no cast; the `enum class` needed one).

Output of Task02/task02.cpp

```
macro value type check : sizeof(BASE_INNER) = 4 (it is just an
int literal)
constexpr array sizeof  : 24 bytes for 3 values
plain enum -> int silently : 1
enum class needs a cast    : 1
DOUBLE_IT(3 + 1) = 5 (expected 8)
constexpr lambda (3 + 1) = 8
peak surcharge on Rs 20 = 400 paise
```

2.4 Naming rules and conventions

Rule (compiler enforces)	Convention (team style used here)
Letters, digits, underscore; cannot start with a digit	Types and classes: `SmartCard`, `FareTable` (PascalCase)
Case sensitive: `fare` and `Fare` differ	Functions and variables: `canTravel`, `balance_` (camelCase, trailing _ for members)

Rule (compiler enforces)	Convention (team style used here)
Cannot be a keyword (`class`, `int`)	Constants: `MIN_BALANCE_PAISE` (UPPER_SNAKE_CASE)
Avoid names starting with _ + capital or containing __ (reserved)	Names say what and unit: `BASE_PAISE`, not `BASE`

2.5 Worked example: a fare revision made in one place

The first version typed the fare numbers inside several functions. A revision (say the Outer zone rises from Rs 30 to Rs 35) meant finding every copy. The table shows the legacy pattern and the corrected pattern side by side.

Legacy versus revised

```
// LEGACY: the number is repeated; one copy is easily missed
long long fareGate(int zone)     { if (zone == 2) return 3000;
... }
long long fareReport(int zone)  { if (zone == 2) return 3000;
... }      // second copy
long long fareRefund(int zone)   { return zone == 2 ? 3000 :
1000; }      // third copy

// REVISED: one table, every function reads it
constexpr long long BASE_PAISE[] = {1000, 2000, 3500};
// the ONLY edit for Rs 35
long long fareAny(Zone z) { return
BASE_PAISE[static_cast<int>(z)]; }
```

Fault in the first version	Cause	Fix in the new design
Fare values typed in several places	No single source of truth	One `constexpr` table, used everywhere
Peak check inconsistent at 10 AM and 8 PM	`<=` in one copy and `<` in another	One `isPeakHour()` with named constants and half-open windows

Fault in the first version	Cause	Fix in the new design
Fares off by a paisa	`double` rupees and `float` percentages	`long long` paise and integer percentages
Negative balance appeared	Balance writable by any function	Private balance with checked `deduct()`

2.6 Literal suffixes and the type of a constant

A literal has a type of its own, and that type decides what the surrounding expression does. The suffix is part of the token.

Literal	Type	Why it matters in this program
`1000`	`int`	Fine for a fare; the result of `int * int` is still `int` and could overflow in a total
`5000LL`	`long long`	Forces 64-bit arithmetic in `balance - fare >= 5000LL`
`20`	`int`	A percentage; `fare * 20 / 100` stays integer
`0.2`	`double`	Avoid: `fare * 0.2` brings a binary fraction into money
`'A'`	`char`	A one-letter category code
`true`	`bool`	A peak flag
`"Fare Rs "`	`const char[9]`	A string literal printed by the report

Alternative Concept and Complexity Analysis

Alternative: write the fares as macros:`#define FARE_INNER 1000`, `#define FARE_MIDDLE 2000` and plain `int` zone codes. The program still works, but the compiler no longer checks anything about these names.

Aspect	`#define` macros + int zones	`constexpr` + `enum class`
Type checking	None. `FARE_INNER` is only text; a macro used as `FARE_INNER / 2` silently becomes integer division.	Typed (`long long`); mixing with a `Zone` is a compile error.
Scope	Whole file after the definition; cannot be hidden in a namespace or class.	Normal C++ scope rules.
Macro expansion bug (measured)	`DOUBLE_IT(3 + 1)` printed **5**, expected 8.	`constexpr` function printed **8**.
Memory	Macro: no storage (literal is inside the instructions). Each use may embed a copy.	`constexpr` scalars normally need no storage either; the 3-value array is 24 bytes (`sizeof` = 24 in the run above).
Running time	No difference.	No difference: both are folded at compile time. O(1).
Compile time	Slightly faster to preprocess.	Negligible extra.

Conclusion: memory and running time are the same, but only the `constexpr` and `enum class` version lets the compiler catch mistakes. It is the chosen design.

TASK 3 - Choose Data Types and Check Type Compatibility

Bloom's Level: K5 - Evaluate | Marks: 4

3.1 Data type chosen for every value

The sizes below were printed by `Task03/task03.cpp` on this system (64-bit Linux, g++ 13.3.0).

Value	Type	size of	Range	Reason
Fare, balance, minimum balance (paise)	`long long`	8	-9.22e18 to 9.22e18	Exact integers; large headroom for totals
Peak surcharge %, concession %	`int`	4	-2,147,483,648 to 2,147,483,647	Small whole numbers
Hour of day	`int`	4	0-23 used	Natural type for arithmetic and comparison
Zone number	`enum class Zone`	4	Inner, Middle, Outer	Closed set, type safe
Peak flag, concession flag	`bool`	1	true / false	Truth value
Passenger category code	`char`	1	'G', 'S', 'N'	One letter is enough
Journeys counted in a day	`long long`	8	up to 9.22e18	3,000,000 fits in `int`, but growth is expected
Day's collection (paise)	`long long`	8	up to 9.22e18	6,000,000,000 does not fit in `int`
Rupees shown to the operator	`double`	8	about 15 digits	Display only; never accumulated

Value	Type	size of	Range	Reason
Local variable with obvious type	`auto`	-	deduced	Used only where the type is clear (`auto f = fare(z, p);`)

Output of Task03/task03.cpp (sizes, ranges from <climits>, precision and overflow)

```
---- sizeof on this system ----
bool          1
char          1
int           4
unsigned      4
long long     8
float         4
double        8
---- ranges from <climits> ----
INT_MIN  = -2147483648
INT_MAX  = 2147483647
UINT_MAX = 4294967295
LLONG_MIN= -9223372036854775808
LLONG_MAX= 9223372036854775807
---- precision problem: 0.10 added 10 times ----
double sum = 0.99999999999999989
sum == 1.0 ? false
```

3.2 Why money must be integer paise, not double rupees

A `double` stores numbers in binary. 0.10 has no exact binary form (it is a repeating fraction, like 1/3 in decimal), so the stored value is slightly wrong and the error is carried into every addition.

- **Evidence from the run:** adding 0.10 ten times gives `0.99999999999999989`, and `sum == 1.0` is **false**. Adding 10 paise ten times as integers gives exactly 100.
- `float` is worse: `0.1f` is stored as `0.1000000015`, so `3 * 0.1f` is `0.3000000119`.
- This is the 'fare off by a paisa' fault in the first version. Integer paise cannot drift: every addition and subtraction is exact.
- Percentages stay exact too: 20% of 3000 paise is `3000 * 20 / 100 = 600`. For fares that are multiples of 5 paise the division has no remainder; otherwise integer division truncates, and the rule (truncate or round) must be stated once.

3.3 Type compatibility: what the compiler does in one expression

Expression	Conversion applied	Result
`3 + 2.5`	`int` converted to `double`	5.5 (double)
`true + 1`	`bool` promoted to `int` (true = 1)	2 (int)
`'A' + 1`	`char` promoted to `int`	66 (int)
`7 / 2`	both `int`: integer division, no conversion	3
`7 / 2.0`	`int` converted to `double` first	3.5
`fare * 20 / 100` (`long long` and `int`)	`int` converted to `long long`	`long long`
`bool b = 5;`	non-zero `int` becomes `true`	true

The rule is the **usual arithmetic conversion**: both operands are brought to the 'wider' type (`int` < `long long` < `float` < `double`), and anything smaller than `int` is first promoted to `int`. Task 4 studies the surprising cases in detail.

3.4 Ranges and the day's collection: is int enough?

`INT_MAX` is 2,147,483,647 paise, which is only **Rs 2,14,74,836**. One day of the whole network at an average fare of Rs 20 is 3,000,000 x 2,000 = **6,000,000,000 paise**, nearly three times `INT_MAX`.

Quantity	Value (paise)	Fits in int?	Fits in long long?
One station, 20,000 journeys x Rs 20	40,000,000	Yes	Yes
One line, 500,000 journeys x Rs 20	1,000,000,000	Yes (barely)	Yes
Whole network, 3,000,000 journeys x Rs 20	6,000,000,000	**No - overflows**	Yes

The run shows what overflow does: 6,000,000,000 squeezed into 32 bits became **1,705,032,704** (the value modulo 2^32), with no error message. Signed overflow is undefined behaviour in C++; the demonstration used `unsigned`, whose wrap-around is defined. **Resolution:** use `long long` for every total.

3.5 Converting and displaying money safely

Paise are stored; rupees appear only on the screen. Two small functions keep the conversion in one place, and `auto` is used only where the type is obvious.

Money helpers

```
constexpr long long toPaise(long long rupees) { return rupees * 100; }
void printMoney(long long paise)                          // Rs 36.00 for 3600
{
    std::cout << "Rs " << paise / 100 << '.' << std::setw(2) << std::setfill('0') << paise % 100
              << std::setfill(' ');
}
auto fare   = 3600LL;      // deduced long long (suffix LL)
auto zoneNo = 2;           // deduced int
auto rate   = 0.2;          // deduced double - do NOT use for money
```

Mistake	Why it is wrong	Correct form
`int total = 0;` for a day's collection	Overflows at 2,147,483,647 paise	`long long total = 0;`
`double fare = 36.00;`	Binary rounding error accumulates	`long long fare = 3600;`
`unsigned balance` for a balance	`balance - fare` wraps to a huge positive value instead of going negative	Signed `long long`; test before subtracting
`char hour` for the hour	Prints as a letter and is unclear	`int hour`
`auto x = 1000;` then adding a large total	`x` is `int`, not `long long`	`auto x = 1000LL;` or an explicit type

Alternative Concept and Complexity Analysis

Alternative: store money as `double` rupees (for example 36.00) instead of `long long` paise.

Alternative (not recommended)

```
double balance = 100.00;                          // alternative design
double f = 10.00;  balance -= f;                  // each step may add a tiny error
// check: balance - f >= 50.0 can be wrong when balance - f is 49.999999999999996
```

Measure	long long paise	double rupees
Time per addition	O(1); 0.07 s for 200 million adds (measured)	O(1); 0.59 s for 200 million adds (measured, same loop)
Space per value	8 bytes	8 bytes
Error after adding Rs 0.10 n = 1,000 times	0	-1.4e-12
n = 100,000	0	1.9e-08

Measure	long long paise	double rupees
n = 3,000,000	0	**1.96e-06** (error keeps growing with n)
Exact comparison `balance - fare >= 5000`	Always correct	Unreliable at the Rs 50 boundary

The timing ratio is for a tight loop on one machine and will vary; the key point is that both are O(1) and 8 bytes, but only paise is exact. The error figures come from `Task10/task10.cpp`.

Conclusion: `long long` paise gives the same time and space with zero error, so double rupees are rejected for storage.

TASK 4 - Control Implicit Conversions

Bloom's Level: K5 - Evaluate | Marks: 4

4.1 Where C++ converts values without being told

Situation	Example	What happens	Risk
Integer division	`7 / 2`	Both operands `int`: result is 3, the .5 is discarded	Fare percentages lose paise
int to double	`double c = a / b;`	Division is done in `int` first, then 3 is converted to 3.0	Looks like real division but is not
Promotion	`ch + 1` with `char ch`	`char` promoted to `int`: prints a number, not a letter	Surprising output
int to bool	`bool flag = 5;`	Any non-zero value becomes `true` (prints 1)	Value information is lost
Signed to unsigned	`s < u` with `int s=-1`	`s` converted to `unsigned`: -1 becomes 4,294,967,295	Wrong comparison
Narrowing	`int x = 3.7;`	Truncates to 3, accepted silently with =	Lost fraction
long long to int	`int n = bigTotal;`	Value wraps or truncates	Overflow of money

4.2 Predict the Output

The program below is the one given in the assignment (`Task04/task04_predict.cpp`).

Program given in the assignment

```
#include <iostream>
int main()
{
 int a = 7, b = 2;
 double c = a / b; double d = static_cast<double>(a) / b;
 std::cout << c << ' ' << d << '\n';
 char ch = 'A' + 2;
```

```
 std::cout << ch << ' ' << ch + 1 << '\n';
 bool flag = 5;
 std::cout << flag << ' ' << std::boolalpha << flag << '\n';
 unsigned int u = 1;
 int s = -1;
 std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (written from the C++ conversion rules)

```
3 3.5
C 68
1 true
s >= u
```

Actual output (compiled with -std=c++17 -Wall -Wextra)

```
3 3.5
C 68
1 true
s >= u
```

Prediction and actual output agree. Explanation, line by line:

Output line	Conversion that took place	Why
`3 3.5`	**Integer division**, then int to double for `c`	`a / b` is `7 / 2` with both `int`, so the result is 3. Assigning 3 to a `double` gives 3.0, printed as `3`. For `d`, `static_cast<double>(a)` makes the left operand `double`, so `b` is converted and the division is 3.5.
`C 68`	**char to int promotion**	`'A' + 2` is 67 stored in a `char`, which is `'C'` and prints as a letter. In `ch + 1` the `char` is promoted to `int`, so the sum is the **number** 68.

Output line	Conversion that took place	Why
`1 true`	**int to bool**	5 is non-zero, so `flag` is `true`. A `bool` prints as `1` by default; after `std::boolalpha` it prints `true`.
`s >= u`	**signed to unsigned**	Comparing `int` with `unsigned int` converts `s` to unsigned. -1 becomes 4,294,967,295, which is larger than 1, so `s < u` is false.

Compiler warning produced

Compile output with -Wall -Wextra

```
=== COMPILE Task04/task04_predict ===
Task04/task04_predict.cpp: In function 'int main()':
Task04/task04_predict.cpp:13:18: warning: comparison of
integer expressions of different signedness: 'int' and
'unsigned int' [-Wsign-compare]
13 |  std::cout << (s < u ? "s < u" : "s >= u") << '\n';
|                  ~~^~~
compile exit 0
```

`-Wsign-compare` warns about exactly the last line. It is the compiler pointing at the surprising result.

4.3 Narrowing, brace initialisation and static_cast

Brace initialisation forbids conversions that can lose information. The statement `int x{3.7};` is rejected:

Compile output for Task04/narrow.cpp

```
Task04/narrow.cpp: In function 'int main()':
Task04/narrow.cpp:3:11: error: narrowing conversion of
'3.7000000000000002e+0' from 'double' to 'int' [-Wnarrowing]
3 |     int x{3.7};   // narrowing double -> int inside braces
|           ^~~
exit 1
```

- `int x = 3.7;` compiles and silently stores 3. `int x{3.7};` is a compile-time error. If truncation is intended, write `int x{static_cast<int>(3.7)};` so the reader can see it.
- Use `static_cast` when the conversion is deliberate: money to double for display (`static_cast<double>(paise) / 100`), `long long` to `int` after a range check, or an enum to its number.
- Do **not** use a cast to silence a warning you do not understand: a cast says the programmer takes responsibility.

4.4 Corrected code

Source: Task04/task04_fixed.cpp (every conversion explicit)

```
// Task 4 - corrected version: every conversion is explicit,
brace-initialised
#include <iostream>
int main()
{
int a{7}, b{2};
double c{static_cast<double>(a) / b};   // 3.5, not 3
double d{static_cast<double>(a / b)};   // 3, shows integer
division on purpose
std::cout << c << ' ' << d << '\n';

char ch{static_cast<char>('A' + 2)};
std::cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';

bool flag{true};                        // no silent int ->
bool
std::cout << flag << ' ' << std::boolalpha << flag << '\n';

unsigned int u{1};
int s{-1};
// compare as signed values: convert u to long long explicitly
```

```
std::cout << (static_cast<long long>(s) < static_cast<long
long>(u) ? "s < u" : "s >= u") << '\n';

// int bad{3.7};   // would be a compile error: narrowing
double -> int
int ok{static_cast<int>(3.7)};          // explicit, intended
truncation
std::cout << ok << '\n';
}
```

Output of the corrected program

```
3.5 3
C 68
1 true
s < u
3
```

Differences from the original are deliberate: line 1 now shows `c = 3.5` (cast before dividing) and `d = 3` (integer division made explicit), and the signed/unsigned comparison is done in `long long`, so `-1 < 1` is correctly **true**. The program now compiles with **no warnings**.

Alternative Concept and Complexity Analysis

Alternative: write every conversion explicitly with `static_cast` and use brace initialisation everywhere, instead of relying on implicit conversions.

Measure	Implicit conversions	Explicit `static_cast` + `{}`
Running time	O(1) per conversion	O(1) per conversion; the compiler normally produces the same machine instruction
Memory	No extra storage	No extra storage (a cast is not a variable)

Measure	Implicit conversions	Explicit `static_cast` + `{}`
Errors caught at compile time	Only some, as warnings (`-Wsign-compare`)	Narrowing is a **hard error**; signed/unsigned comparisons are visible and handled
Errors left for run time	Wrong totals, wrong comparisons, silent truncation	Only genuine range problems (checked by an explicit `if`)
Readability	Short but hides intent	Longer, but each cast documents a decision

Conclusion: explicit conversions cost nothing at run time and move a whole class of bugs from run time to compile time. They are adopted in the final design.

TASK 5 - Write Correct Expressions with the Right Precedence

Bloom's Level: K5 - Evaluate | Marks: 4

5.1 The three expressions of the fare system

Expressions used in the program

```
constexpr bool isPeakHour(int hour)
{
return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
// peak check
}

long long surcharge = base * PEAK_SURCHARGE_PERCENT / 100;
// surcharge
long long fare      = isPeakHour(hour) ? base + surcharge :
base;    // conditional operator
if (isConcession) fare -= fare * CONCESSION_PERCENT / 100;
// concession, compound assignment
```

5.2 Precedence and associativity (high to low) for the operators used

Level	Operators	Associativity	Example in this system
1	`() []` postfix `++ --`	left to right	`BASE_PAISE[z],x++`
2	prefix `++ --`, unary `-`, `!`, casts	right to left	`++y,` `static_cast<double>(a)`
3	`* / %`	left to right	`base * PEAK / 100`
4	`+ -`	left to right	`base + surcharge`
5	`< <= > >=`	left to right	`hour >= 8,hour < 10`
6	`== !=`	left to right	`zone == 2`
7	`&&`	left to right	`hour >= 8 && hour < 10`
8	`||`	left to right	`(...) || (...)`
9	`?:`	right to left	`peak ? a : b`

Level	Operators	Associativity	Example in this system
10	= += -=	right to left	`fare -= x`

Reading `base * PEAK_SURCHARGE_PERCENT / 100`: `*` and `/` have the same level and associate left to right, so it is `(base * 20) / 100`. Multiplying first keeps the intermediate value exact (3000 * 20 = 60000 before the division). Writing `base * (20 / 100)` would give `base * 0` because `20 / 100` is integer division and equals 0.

The conditional operator has lower precedence than +, so `x ? a + b : c` parses as `x ? (a + b) : c`. **Compound assignment** `fare -= fare * 50 / 100` evaluates the right side completely first and then subtracts it; the operator is right-associative, so `a = b = 0` assigns right to left.

5.3 Why the parenthesised peak check is better

`&&` has higher precedence than `||`, so the two forms mean the same thing:

Equivalent forms

```
(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)     //
clear: two windows
hour >= 8 && hour < 10  ||  hour >= 17 && hour < 20      //
equivalent, but the reader must recall the rule
```

The parenthesised version is correct **and** readable. The danger is a maintainer editing the unparenthesised line and changing the grouping by accident (`hour >= 8 && (hour < 10 || hour >= 17) && hour < 20` is always false). Compilers warn about mixed `&&`/`||` without parentheses under `-Wparentheses`.

5.4 x++ versus ++x, and why i = i++ is a mistake

- `x++` (postfix) yields the **old** value, then increments. `++x` (prefix) increments, then yields the **new** value.
- With `x = 5, y = 3`: `x++` is 5 (x becomes 6) and `++y` is 4 (y becomes 4), so `x++ + ++y` is 9.

- `i = i++` stores the old value back, so the increment is lost and the statement does nothing useful. Before C++17 the result was **undefined behaviour** (two modifications of `i` without sequencing); since C++17 the right side is sequenced first, so `i` keeps its old value. Either way it is a mistake - write `i++;` or `i = i + 1;`.

5.5 Division and remainder with negative operands

Since C++11, `/` **truncates toward zero** and `%` takes the sign of the left operand, so that `(a / b) * b + a % b == a` always holds. For `-10 / 3` the result is -3 and `-10 % 3` is -1 (check: -3 * 3 + (-1) = -10). In the fare system, `%` is used on non-negative paise only (`f % 100` for the paise part), so no negative remainder is expected; a negative fare would be rejected first.

5.6 Predict the Output

Program given in the assignment

```
#include <iostream>
int main()
{
int x = 5, y = 3;
std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
int z = x++ + ++y;
std::cout << z << ' ' << x << ' ' << y << '\n';
std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' '
<< -10 % 3 << '\n';
int k = 0;
for (int i = 0; i < 10; i += 3) k += i;
std::cout << k << '\n';
}
```

My predicted output (written from precedence and sequencing rules)

```
11 16
9 6 4
6 4
```

```
3 1 -3 -1
18
```

Actual output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Prediction and actual output agree. Operator-by-operator explanation:

Line	Operators and order of evaluation	Result
`x + y * 2 (x + y) * 2`	`*` before `+` (higher precedence): 3 * 2 = 6, then 5 + 6. With parentheses `+` first: 8, then 8 * 2.	11 16
`z = x++ + ++y`	Postfix `x++` gives 5 and sets x = 6. Prefix `++y` sets y = 4 and gives 4. Then `+`: 5 + 4. Finally `=` stores 9.	9 6 4
`(x > y ? x : y)`	Parentheses force `?:` to be evaluated as a unit. x = 6 > y = 4 is true, so the value is x = 6.	6
`x % 4 * 2`	`%` and `*` same level, left to right: (6 % 4) = 2, then 2 * 2.	4
`10 / 3, 10 % 3`	Integer division and remainder.	3 1
`-10 / 3, -10 % 3`	Unary minus first (level 2), then truncation toward zero; remainder takes the sign of -10.	-3 -1
`for (i = 0; i < 10; i += 3) k += i;`	`i` takes 0, 3, 6, 9 (stops at 12). `+=` adds each: 0 + 3 + 6 + 9.	18

Alternative Concept and Complexity Analysis

Alternative: replace the combined `&&` / `||` expression with a lookup table indexed by the hour.

Alternative design

```
constexpr bool PEAK[24] = {
false,false,false,false,false,false,false,false,    // 0-7
true, true, false,false,false,false,false,false,    // 8-15
(8,9 peak)
false,true, true, true, false,false,false,false }; // 16-23
(17,18,19 peak)
bool peak = PEAK[hour];       // hour must be checked to be 0-
23 first
```

Measure	**Expression `(h>=8&&h<10)||(h>=17&&h<20)`**	**Table `PEAK[hour]`**
Time per check	O(1): 2 to 4 comparisons (short-circuit)	O(1): one indexed load
Measured, 300 million checks	0.386 s	0.319 s
Extra space	none	24 bytes
Changing the peak window	Edit 2 numbers	Edit several array entries (error-prone)
Bounds risk	None: any int is handled	`hour` outside 0-23 reads outside the array unless checked
Readability	Shows the windows directly	Shows a pattern of 24 flags

Conclusion: both are O(1). The table saves a few comparisons (about 17% faster in this loop) at the cost of 24 bytes and a bounds check. The expression is kept because it states the rule and needs no bounds protection; the table is worthwhile only if the rule becomes irregular.

TASK 6 - Design the Control Structures

Bloom's Level: K5 - Evaluate | Marks: 4

6.1 Choosing the decision structure

Structure	Use it when	In the fare system
`if-else`	Conditions are ranges or combinations	Peak check (`hour` ranges), `canTravel`, concession
`switch`	One integer or enum value selects among fixed cases	Pick the base fare from the zone
`?:`	A simple two-way choice that produces a value	`peak ? base + surcharge : base`

A `switch` on an `enum class` is the best fit for the zone: each case is a constant, the compiler can build a jump table, and `-Wswitch` (part of `-Wall`) warns if a new enumerator has no case. A **missing `break`** makes execution fall through into the next case, and when **no case matches** and there is no `default`, the switch is simply skipped, so the variable it should have set keeps its initial value. The program therefore initialises `base = 0` and adds a `default` branch so that an invalid zone is visible.

6.2 Choosing the loop

Loop	Strength	Best used for
`for (int i = 0; i < n; ++i)`	Index available; known count	Needing the position (journey number)
range-based `for (const Trip& t : trips)`	No index errors, shortest to read	**Processing every journey of the day**
`while (cin >> hour)`	Count unknown, driven by input	Reading gate events until the end of input

The range-based `for` is best for a stored day's journeys because it cannot run past the end of the array. The `while` loop is best for live gate input. `continue` skips a journey that is invalid

(unknown zone, bad hour) and moves to the next, and `break` stops the whole batch when a fatal condition occurs (for example the card reader fails).

break and continue in the daily loop

```
for (const Journey& j : journeys) {
if (j.hour < 0 || j.hour > 23) { ++rejected; continue; }   //
skip invalid, keep processing
long long f = fare(j.zone, isPeakHour(j.hour));
if (!card.deduct(f)) { ++denied; continue; }               //
low balance: record and move on
total += f;
if (readerFailed()) break;                                  //
fatal: stop the batch
}
```

6.3 Pseudocode for the fare computation

Structured pseudocode (header, INPUT/OUTPUT, numbered steps, indentation for blocks). Adapt the headings to the SIMATS Engineering template if your copy of the standard uses different keywords.

Pseudocode

```
ALGORITHM ComputeFare
INPUT   zone (Inner | Middle | Outer), hour (0..23)
OUTPUT  fare in paise (integer)
CONSTANTS BASE_PAISE = [1000, 2000, 3000],
PEAK_SURCHARGE_PERCENT = 20
1.  base <- 0
2.  SELECT zone
CASE Inner  : base <- BASE_PAISE[0]
CASE Middle : base <- BASE_PAISE[1]
CASE Outer  : base <- BASE_PAISE[2]
OTHERWISE   : base <- 0                    // invalid zone,
no fare
```

```
3.  peak <- (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour
< 20)
4.  IF peak THEN base <- base + base * PEAK_SURCHARGE_PERCENT
/ 100      // integer arithmetic
5.  RETURN base
ALGORITHM CanTravel
INPUT balance, fare            OUTPUT true or false
1.  RETURN (balance - fare) >= MIN_BALANCE_PAISE
```

6.4 Complete the Code (TODO 1 and TODO 2) - completed program

Source: Task06/task06.cpp (TODO 1 and TODO 2 completed; peak-boundary test added)

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs
20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must
remain on the card

// Peak = 8:00-9:59 and 17:00-19:59 (start inclusive, end
exclusive)
constexpr bool isPeakHour(int hour)
{
return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

long long fare(Zone z, bool peak)
{
long long base = 0;
```

```
switch (z) {
case Zone::Inner:  base = BASE_PAISE[0]; break;
case Zone::Middle: base = BASE_PAISE[1]; break;
case Zone::Outer:  base = BASE_PAISE[2]; break;
default:           base = 0;             break; // unknown
zone: no fare
}
if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100; //
integer arithmetic only
return base;
}

bool canTravel(long long balance, long long f)
{
return balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
long long balance = 10000; // Rs 100.00 in paise
struct Trip { Zone zone; bool peak; };
Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
{Zone::Middle, false}};
for (const Trip& t : trips) {
long long f = fare(t.zone, t.peak);
if (canTravel(balance, f)) {
balance -= f;
std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
<< std::setfill('0') << f % 100 << std::setfill(' ');
} else {
std::cout << "Denied (low balance)"; }
std::cout << " balance Rs " << balance / 100 << '\n';
}
```

```
std::cout << "\n--- peak boundary check (hour -> peak?) ---
\n";
int hours[] = {7, 8, 9, 10, 16, 17, 19, 20};
for (int h : hours)
std::cout << std::setw(2) << h << ":00 -> " << (isPeakHour(h)
? "PEAK" : "off-peak") << '\n';

std::cout << "\n--- journeys at exactly 10 AM and 8 PM, Middle
zone ---\n";
for (int h : {10, 20}) {
long long f = fare(Zone::Middle, isPeakHour(h));
std::cout << h << ":00 Middle fare = " << f << " paise\n";
}
return 0;
}
```

Output (first three lines match the expected output in the assignment)

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

--- peak boundary check (hour -> peak?) ---
 7:00 -> off-peak
 8:00 -> PEAK
 9:00 -> PEAK
10:00 -> off-peak
16:00 -> off-peak
17:00 -> PEAK
19:00 -> PEAK
20:00 -> off-peak

--- journeys at exactly 10 AM and 8 PM, Middle zone ---
10:00 Middle fare = 2000 paise
```

```
20:00 Middle fare = 2000 paise
```

Expected (assignment)	Actual	Match
Fare Rs 10.00 balance Rs 90	Fare Rs 10.00 balance Rs 90	Yes
Fare Rs 36.00 balance Rs 54	Fare Rs 36.00 balance Rs 54	Yes
Denied (low balance) balance Rs 54	Denied (low balance) balance Rs 54	Yes

Check of the third line: Middle zone off-peak costs 2000 paise; 5400 - 2000 = 3400, which is below 5000, so the journey is denied. **Peak boundary:** 8:00 is PEAK, 10:00 is off-peak, 17:00 is PEAK, 20:00 is off-peak. A Middle-zone journey at exactly 10 AM and at exactly 8 PM both cost 2000 paise (no surcharge). The 10 AM / 8 PM inconsistency is gone because the end of each window is exclusive.

Alternative Concept and Complexity Analysis

Alternative: array indexed by zone

```
constexpr long long FARE[] = {1000, 2000, 3000};
long long base = FARE[static_cast<int>(z)];        // replaces
the whole switch
```

Measure	`switch`	Array `FARE[zone]`
Time per lookup	O(1) (compiler may emit a jump table)	O(1) (one indexed load)
Measured, 300 million lookups	0.279 s	0.267 s (about 4% faster; effectively equal)
Space	none for the cases (code only)	24 bytes (3 x `long long`)
Invalid zone	`default` branch catches it	Reads outside the array unless a range check is added
Adding a fourth zone	New enumerator, new `case`, new array entry	New enumerator and one new array entry
Missed update warning	`-Wswitch` warns about a missing `case`	No warning; the array is silently too short

Conclusion: speed is the same. The `switch` is safer for three zones (the compiler checks completeness); the array is shorter and scales better to many zones. Task 12 moves to the array and adds an explicit range check.

TASK 7 - From Procedural to Object-Oriented

Bloom's Level: K5 - Evaluate | Marks: 4

7.1 Procedural version (separate variables and free functions)

Source: Task07/task07_procedural.cpp

```
// Task 7 - procedural version: global variable + free
functions
#include <iostream>

constexpr long long MIN_BALANCE_PAISE = 5000;
long long balancePaise = 10000;          // global: any
function can change it

bool canTravel(long long fare) { return balancePaise - fare >=
MIN_BALANCE_PAISE; }
void deduct(long long fare)    { balancePaise -= fare; }

int main()
{
long long fares[] = {1000, 3600, 2000};
for (long long f : fares) {
if (canTravel(f)) { deduct(f); std::cout << "Paid " << f << "
balance " << balancePaise << '\n'; }
else              { std::cout << "Denied " << f << " balance "
<< balancePaise << '\n'; }
}
balancePaise = -999;                 // nothing stops this bug
std::cout << "After accidental write, balance = " <<
balancePaise << '\n';
return 0;
}
```

Output

```
Paid 1000 balance 9000
Paid 3600 balance 5400
Denied 2000 balance 5400
After accidental write, balance = -999
```

The three journeys behave correctly, but the last line shows the weakness: **any statement in the program** may write `balancePaise = -999`, and the compiler accepts it. This is exactly how a negative balance can appear on a card.

7.2 Object-oriented version (SmartCard with a private balance)

Source: Task07/task07_oop.cpp

```
// Task 7 - object-oriented version: SmartCard keeps the
balance private
#include <iostream>

class SmartCard {
public:
static constexpr long long MIN_BALANCE_PAISE = 5000;

explicit SmartCard(long long openingPaise) :
balance_{openingPaise < 0 ? 0 : openingPaise} {}

bool canTravel(long long fare) const { return fare >= 0 &&
balance_ - fare >= MIN_BALANCE_PAISE; }

bool deduct(long long fare)            // returns false and
changes nothing if not allowed
{
if (!canTravel(fare)) return false;
balance_ -= fare;
return true;
}
```

```
bool recharge(long long paise)         // negative recharge is
rejected
{
if (paise <= 0) return false;
balance_ += paise;
return true;
}

long long balance() const { return balance_; }

private:
long long balance_;                    // cannot be touched
from outside
};

int main()
{
SmartCard card{10000};
long long fares[] = {1000, 3600, 2000};
for (long long f : fares) {
if (card.deduct(f)) std::cout << "Paid " << f << " balance "
<< card.balance() << '\n';
else                std::cout << "Denied " << f << " balance "
<< card.balance() << '\n';
}
std::cout << "recharge(-500) accepted? " << std::boolalpha <<
card.recharge(-500)
<< ", balance " << card.balance() << '\n';
// card.balance_ = -999;   // compile error: 'balance_' is
private
return 0;
}
```

Output

```
Paid 1000 balance 9000
Paid 3600 balance 5400
Denied 2000 balance 5400
recharge(-500) accepted? false, balance 5400
```

The journeys give the same results (balances 9000, 5400, denied). The line `card.balance_ = -999;` is commented out because it is a **compile-time error** (`'balance_' is private`). The negative recharge is refused and the balance stays 5400.

7.3 How the class prevents a negative balance

Entry point	Guard	Effect
Constructor	`openingPaise < 0 ? 0 : openingPaise`	A card can never start negative
`canTravel(fare)`	`fare >= 0 && balance_ - fare >= MIN_BALANCE_PAISE`	Refuses negative fares and any fare that would leave less than Rs 50
`deduct(fare)`	Calls `canTravel` first; changes nothing if false	Balance only decreases when safe, so it stays >= 5000 (>= 0)
`recharge(p)`	`p <= 0` returns false	A negative recharge cannot reduce the balance
`balance_`	`private`	No other code can bypass the three guards above

This is an **invariant**: every public member function starts with the balance valid and ends with the balance valid. The proof is short because only three functions can change one variable.

7.4 Which version is easier to change when the minimum balance changes?

Question	Procedural	Object-oriented
Where is the number written?	Constant at the top, but the check is in a free function that other code may duplicate	One `static constexpr` inside `SmartCard`
Can other code bypass the rule?	Yes (`balancePaise` is global and writable)	No (`balance_` is private)
Edits needed	1 constant, plus a search for every place that compares the balance	1 constant
Risk after the change	A forgotten copy keeps the old rule	None; the rule exists once

Both versions can change the constant in one line, so the honest difference is **confidence**: in the OO version one edit is provably enough; in the procedural version it is enough only if nobody wrote the comparison elsewhere.

Alternative Concept and Complexity Analysis

Alternative: public balance

```
class SmartCard {
public:
    long long balance;                    // public data member:
alternative design
};
SmartCard c{10000};
c.balance = -999;                          // compiles and runs:
the rule is broken
c.balance -= 9000;                         // bypasses the Rs 50
minimum
```

Measure	Private + `deduct()`/`canTravel()`	Public data member
Time per journey	O(1); one extra function call (inlined by the compiler)	O(1)
Space per card	8 bytes	8 bytes
Rule R1 (>= Rs 50 remains)	Enforced in one place	Any code can break it
Rule R2 (never negative)	Enforced	Any code can break it, as the last line of the procedural run shows
Rule R7 (negative recharge refused)	Enforced by `recharge()`	Can be broken with `balance += -500`

Conclusion: time and space do not change, but with a public member **three of the system's rules can be broken by any part of the program**. The private-member design is chosen.

TASK 8 - Read and Print Data Safely

Bloom's Level: K5 - Evaluate | Marks: 4

8.1 How cin fails, and how to recover

When `cin >> hour` meets letters instead of digits, it cannot convert them. Three things happen: `hour` is set to 0 (C++11 and later), the stream's **fail state** is set, and the bad characters **stay in the input buffer**. Every later `cin >>` then fails immediately, so a naive loop spins forever. Recovery needs two steps:

Step	Call	Purpose
1. Detect	`if (!(cin >> x))` or `cin.fail()`	Learn that the read failed
2. Clear the state	`cin.clear()`	Remove the fail flag so reading can resume
3. Discard the bad input	`cin.ignore(numeric_limits<streamsize>::max(), '\n')`	Throw away the rest of the line
4. Validate the range	`if (x < lo || x > hi)`	Letters are not the only bad input; 25 is not an hour
5. End of input	`cin.eof()`	Stop instead of looping when the input ends

8.2 Safe input program

Source: Task08/task08_cin.cpp

```
// Task 8 - safe input with cin >>, cin.fail(), cin.clear(),
cin.ignore()
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>

// Reads an int in [lo, hi]; keeps asking until valid. Returns
false on end of input.
bool readInt(const char* prompt, int lo, int hi, int& out)
```

```
{
while (true) {
std::cout << prompt;
if (std::cin >> out) {
if (out >= lo && out <= hi) return true;
std::cout << "  out of range (" << lo << "-" << hi << "), try
again\n";
} else {
if (std::cin.eof()) return false;
std::cout << "  not a number, try again\n";
std::cin.clear();
// reset the error state
std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n'); // discard bad line
}
}
}

int main()
{
int zone = 0, hour = 0;
if (!readInt("Zone (0=Inner,1=Middle,2=Outer): ", 0, 2, zone))
return 1;
if (!readInt("Hour (0-23): ", 0, 23, hour)) return 1;

constexpr int BASE[] = {1000, 2000, 3000};
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour <
20);
long long fare = BASE[zone];
if (peak) fare += fare * 20 / 100;

std::ostringstream money;                          // build
"24.00" first so it can be padded as one field
```

```
money << fare / 100 << '.' << std::setfill('0') <<
std::setw(2) << fare % 100;

std::cout << "\n+--------------------------+\n";
std::cout << "| JOURNEY REPORT           |\n";
std::cout << "+--------------------------+\n";
std::cout << std::left;
std::cout << "| Zone    : " << std::setw(15) << zone << "|\n";
std::cout << "| Hour    : " << std::setw(15) << hour << "|\n";
std::cout << "| Peak    : " << std::setw(15) << (peak ? "Yes"
: "No") << "|\n";
std::cout << "| Fare Rs : " << std::setw(15) << money.str() <<
"|\n";
std::cout << "+--------------------------+\n";

std::cout << std::fixed << std::setprecision(2) << "As double
rupees: Rs " << fare / 100.0 << '\n';
return 0;
}
```

8.3 Demonstrated recovery from invalid input

Input typed (piped from a file): `abc`, `7`, `1`, `xyz`, `25`, `9`. Letters and out-of-range values are rejected, and the program recovers each time:

Output with invalid input (screenshot-style)

```
Zone (0=Inner,1=Middle,2=Outer):   not a number, try again
Zone (0=Inner,1=Middle,2=Outer):  out of range (0-2), try again
Zone (0=Inner,1=Middle,2=Outer): Hour (0-23):   not a number,
try again
Hour (0-23):   out of range (0-23), try again
Hour (0-23):
+--------------------------+
| JOURNEY REPORT           |
```

```
+--------------------------+
| Zone    : 1              |
| Hour    : 9              |
| Peak    : Yes            |
| Fare Rs : 24.00          |
+--------------------------+
As double rupees: Rs 24.00
```

Zone: `abc` is rejected as not a number, `7` is rejected as out of range, `1` is accepted. Hour: `xyz` is rejected, `25` is out of range, `9` is accepted. Hour 9 is a peak hour, so the Middle-zone fare is Rs 20 + 20% = **Rs 24.00**, printed correctly.

A subtle failure: input such as 2x

`cin >> zone` accepts the `2` and leaves `x` in the buffer, so the **next** read fails. The run with input `2x`, `1`, `9` shows it:

Output for input '2x'

```
Zone (0=Inner,1=Middle,2=Outer): Hour (0-23):   not a number,
try again
Hour (0-23):
+--------------------------+
| JOURNEY REPORT           |
```

The zone was taken as 2, but the hour prompt immediately reported 'not a number' because of the leftover `x`. This is the main weakness of `cin >>` that Section 8.5 fixes.

8.4 Formatting money and the journey report

Manipulator	Effect	Used for
`setw(n)`	Minimum field width for the next item only	Aligning the report columns
`setfill('0')`	Pad character (stays until changed)	Paise part `05` instead of `5`
`fixed`	Fixed-point notation	Display of double rupees

Manipulator	Effect	Used for
`setprecision(2)`	With `fixed`: two digits after the point	`Rs 24.00`
`left` / `right`	Alignment inside the field	Text left, numbers right

The paise are printed from integers (`fare / 100`, `'.'`, `setw(2)` with fill `0`, `fare % 100`), so no double is involved. The report builds the money text with an `ostringstream` first so it can be padded as one field; the last line of the output shows the same value as double rupees (`fixed` + `setprecision(2)`) for comparison.

Alternative Concept and Complexity Analysis

Alternative: read each line with `std::getline()` and convert it with `std::stoi()` inside `try`/`catch`.

Source: Task08/task08_getline.cpp (key function)

```
bool readIntLine(const char* prompt, int lo, int hi, int& out)
{
std::string line;
while (true) {
std::cout << prompt;
if (!std::getline(std::cin, line)) return false;
try {
std::size_t used = 0;
int v = std::stoi(line, &used);
if (used != line.size()) throw std::invalid_argument("trailing
characters");
if (v < lo || v > hi) throw std::out_of_range("range");
out = v;
return true;
} catch (const std::invalid_argument&) {
std::cout << "  invalid input '" << line << "', try again\n";
} catch (const std::out_of_range&) {
std::cout << "  value out of range, try again\n";
}
```

```
}
}
```

Output with input: hello, 2x, 1, 30, 9

```
Zone (0-2):   invalid input 'hello', try again
Zone (0-2):   invalid input '2x', try again
Zone (0-2): Hour (0-23):   value out of range, try again
Hour (0-23): Accepted zone=1 hour=9
```

Measure	**`cin >>`**	**`getline` + `stoi` + `try/catch`**
Time for a line of length L	O(L)	O(L): `getline` copies L characters, `stoi` scans them
Buffer space	Stream's internal buffer	O(L): one `std::string` per line
Bad input `abc`	Fail state; needs `clear()` and `ignore()`	`stoi` throws `invalid_argument`; stream stays healthy
Bad input `2x`	Accepts 2, leaves `x` (breaks next read)	Rejected by the `used != line.size()` check
Huge number	Fail state	`out_of_range` exception
Recovery code	Manual, easy to forget	Automatic: the whole line is already consumed

Conclusion: both are O(L). The `getline` + `stoi` design is more robust because each line is consumed completely and trailing garbage is detected. It is recommended for the gate; `cin >>` with `clear()`/`ignore()` is acceptable for simple tools.

TASK 9 - Implement and Test the Fare Logic

Bloom's Level: K5 - Evaluate | Marks: 4

9.1 Implementation

The complete logic is `computeFare(zone, hour)` (returns -1 for an invalid zone or hour), `isPeakHour(hour)` and `canTravel(balance, fare)`. Money is `long long` paise, the peak windows are half-open, and every percentage uses integer arithmetic. The tests are stored in a table and run by one loop (this is also the alternative design asked for at the end of the task).

Source: Task09/task09.cpp

```
// Task 9 - complete fare logic + table-driven tests
#include <iomanip>
#include <iostream>
#include <string>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

constexpr bool isPeakHour(int hour) { return (hour >= 8 &&
hour < 10) || (hour >= 17 && hour < 20); }

// returns -1 for invalid zone number or invalid hour
long long computeFare(int zoneNo, int hour)
{
if (zoneNo < 0 || zoneNo > 2 || hour < 0 || hour > 23) return
-1;
long long f = BASE_PAISE[zoneNo];
if (isPeakHour(hour)) f += f * PEAK_SURCHARGE_PERCENT / 100;
return f;
```

```
}

bool canTravel(long long balance, long long fare) { return
balance - fare >= MIN_BALANCE_PAISE; }

struct FareCase { const char* name; int zone; int hour; long
long expected; };

int main()
{
FareCase cases[] = {
{"Inner off-peak 12:00",    0, 12, 1000}, {"Middle off-peak
12:00",   1, 12, 2000},
{"Outer off-peak 12:00",    2, 12, 3000}, {"Inner peak 08:30",
0,  8, 1200},
{"Middle peak 18:00",       1, 18, 2400}, {"Outer peak 09:00",
2,  9, 3600},
{"Boundary 08:00 (peak)",   1,  8, 2400}, {"Boundary 10:00
(off)",    1, 10, 2000},
```

9.2 Test plan

Category	Cases	What it proves
Normal	Each zone, peak and off-peak (6 cases)	Base fares and the 20% surcharge are right
Boundary: time	07:xx, 08:00, 10:00, 17:00, 19:00, 20:00	Windows are [8,10) and [17,20); 10 AM and 8 PM are off-peak
Boundary: balance	Balance leaves exactly Rs 50 / one paisa less	>= (not >) in the minimum-balance rule
Invalid	Unknown zone 5, hour 25, negative recharge, letters for the hour	The gate fails safe and never guesses

Category	Cases	What it proves
Hand total	5 journeys with a total worked out by hand	Totals add exactly

9.3 Fare test table (expected versus actual)

Case	Expected (paise)	Actual (paise)	Result
Inner off-peak 12:00	1000	1000	PASS
Middle off-peak 12:00	2000	2000	PASS
Outer off-peak 12:00	3000	3000	PASS
Inner peak 08:30	1200	1200	PASS
Middle peak 18:00	2400	2400	PASS
Outer peak 09:00	3600	3600	PASS
Boundary 08:00 (peak)	2400	2400	PASS
Boundary 10:00 (off)	2000	2000	PASS
Boundary 17:00 (peak)	2400	2400	PASS
Boundary 20:00 (off)	2000	2000	PASS
Boundary 07:59 hr 7	2000	2000	PASS
Boundary 19:00 (peak)	2400	2400	PASS
Invalid zone 5	-1	-1	PASS
Invalid hour 25	-1	-1	PASS

An expected value of -1 means 'rejected as invalid'. All 14 fare cases pass.

9.4 Balance boundary and hand-calculated total

Balance (paise)	Fare (paise)	Balance left	Expected	Actual	Result
6000	1000	5000 (exactly Rs 50)	allowed	true	PASS
5999	1000	4999	denied	false	PASS
8600	3600	5000 (exactly Rs 50)	allowed	true	PASS

Balance (paise)	Fare (paise)	Balance left	Expected	Actual	Result
8599	3600	4999	denied	false	PASS

Journey	Zone	Hour	Peak?	Fare (hand)	Running total
1	Inner	12	No	1000	1000
2	Middle	18	Yes (+20%)	2400	3400
3	Outer	13	No	3000	6400
4	Outer	9	Yes (+20%)	3600	10000
5	Inner	17	Yes (+20%)	1200	**11200** (Rs 112.00)

Hand calculation of the 5-journey total.

Output of Task09/task09.cpp (complete run)

```
Case                      Expected  Actual    Result
Inner off-peak 12:00      1000      1000      PASS
Middle off-peak 12:00     2000      2000      PASS
Outer off-peak 12:00      3000      3000      PASS
Inner peak 08:30          1200      1200      PASS
Middle peak 18:00         2400      2400      PASS
Outer peak 09:00          3600      3600      PASS
Boundary 08:00 (peak)     2400      2400      PASS
Boundary 10:00 (off)      2000      2000      PASS
Boundary 17:00 (peak)     2400      2400      PASS
Boundary 20:00 (off)      2000      2000      PASS
Boundary 07:59 hr 7       2000      2000      PASS
Boundary 19:00 (peak)     2400      2400      PASS
Invalid zone 5            -1        -1        PASS
Invalid hour 25           -1        -1        PASS

--- balance boundary: leaves exactly Rs 50 ---
balance 6000 fare 1000 -> true  PASS
balance 5999 fare 1000 -> false  PASS
```

```
balance 8600 fare 3600 -> true  PASS
balance 8599 fare 3600 -> false  PASS

--- 5 journeys, total calculated by hand = 11200 paise ---
computed total = 11200 paise (Rs 112.00)
hand total     = 11200 paise -> PASS

Passed 19 of 19 tests
```

Invalid cases from the other programs

Invalid case	Where tested	Result
Unknown zone (5)	`computeFare(5, 12)` in Task 9	Returns -1, rejected
Hour 25	`computeFare(0, 25)` in Task 9	Returns -1, rejected
Negative recharge	`card.recharge(-500)` in Task 7	Returns false; balance stays 5400
Letters typed for the hour	`xyz` typed in Task 8	'not a number', re-prompted

Alternative Concept and Complexity Analysis

Alternative: separate hand-written checks instead of a table of cases.

Alternative: hand-written checks

```
if (computeFare(0, 12) != 1000) std::cout << "FAIL inner off-peak\n";
if (computeFare(1, 18) != 2400) std::cout << "FAIL middle peak\n";
if (computeFare(1,  8) != 2400) std::cout << "FAIL 08:00\n";
// ... one new if-statement and one new message for every extra case
```

Measure	Hand-written `if` checks	Table of cases + loop
Time for t tests	O(t)	O(t)
Space	Code grows with t (about 1 line per test)	Table of t records, about 24 bytes each (two `int`, one `long long`, one pointer)
Adding a boundary case	New statement and message	**One new row**
Summary (passed/total)	Must be counted by hand	Counted by the loop (`Passed 19 of 19`)
Risk	Copy-and-paste errors	A wrong expected value in one row only

Conclusion: both run in O(t), but the table is easier to read, extend and audit, so it is the chosen test design.

TASK 10 - Compare Expectations with Execution

Bloom's Level: K5 - Evaluate | Marks: 4

10.1 Expectation

Before running the experiment the expectation was: (1) with whole-rupee fares (Rs 10, 20, 30 and the peak fares Rs 12, 24, 36) double and integer-paise totals would agree because those values are exactly representable; (2) with fractional fares (concession fares such as Rs 8.50 or Rs 10.20) double would drift, and the drift would grow with the number of journeys; (3) an `int` total would overflow for the network.

10.2 Measured totals: double rupees versus integer paise

`Task10/task10.cpp` adds the fares of 20,000, 500,000 and 3,000,000 journeys, cycling through six fares, once in `long long` paise and once in `double` rupees.

Output of Task10/task10.cpp (compiled with -O2 -Wall -Wextra)

```
== Standard fares (whole rupees) ==
Journeys   Paise total      double (rupees)        double*100 -
paise (raw error, paise)
20000      43998600        439986.000000         0.000e+00
500000     1099998600      10999986.000000       0.000e+00
3000000    6600000000      66000000.000000       0.000e+00

== Concession fares (fractional rupees) ==
Journeys   Paise total      double (rupees)        double*100 -
paise (raw error, paise)
20000      37398810        373988.100000         2.280e-06
500000     934998810       9349988.100003        3.448e-04
3000000    5610000000      56100000.000442       4.416e-02

--- error when adding Rs 0.10 n times ---
```

Fare set	Journeys	Exact total (paise)	double total (Rs)	Raw error (paise)
Standard (whole rupees)	20,000	43,998,600	439,986.000000	0
Standard	500,000	1,099,998,600	10,999,986.000000	0
Standard	3,000,000	6,600,000,000	66,000,000.000000	0
Concession (fractional)	20,000	37,398,810	373,988.100000	2.28e-06
Concession	500,000	934,998,810	9,349,988.100003	3.45e-04
Concession	3,000,000	5,610,000,000	56,100,000.000442	**4.42e-02**

Reading the result: for whole-rupee fares double and paise **agree exactly** (difference 0). For fractional fares double is wrong in every case, and the error grows from 2.3e-06 paise at 20,000 journeys to 4.4e-02 paise at 3,000,000 - about 19,000 times larger for 150 times more journeys, so it grows faster than the number of journeys. The error is still far below one paisa, but it makes == comparisons and the Rs 50 boundary test unreliable (the 9,349,988.100003 above shows a stray digit). The integer total is exact in every row.

10.3 Does int overflow for the network total?

Value	Result
Exact network total (3,000,000 x Rs 20)	6,000,000,000 paise
`INT_MAX`	2,147,483,647
Exceeds `int`?	**true**
Value stored in 32 bits	1,705,032,704 (wrong, no error reported)
Journeys of Rs 20 an `int` total can hold	1,073,741 (about one-third of one day's network traffic)

Yes, `int` overflows for the network total. The expectation was confirmed. Even the 500,000-journey line total (1,099,998,600 paise) is only about 51% of `INT_MAX`, so it fits today but leaves little headroom.

10.4 Compiler warnings received with -Wall -Wextra

Warning	Program	Meaning	Action
`-Wsign-compare`: comparison of integer expressions of different signedness: 'int' and 'unsigned int'	Task 4 given program (`s < u`)	`int` is silently converted to unsigned, so -1 becomes 4,294,967,295	Compare as `long long` (Task 4 fixed version): no warning
`-Wunused-variable`: unused variable 'exact'	First draft of Task 10	A variable was computed but never used (dead code)	Variable removed; recompiled clean

All other programs compile with **zero warnings**. The first warning predicted a real behavioural bug (Task 4); the second revealed leftover code. Both are reasons to compile every program with `-Wall -Wextra`.

10.5 Reasons for the differences

- **Binary fractions:** 8.50, 10.20 and 25.50 are not all exact in binary, so each addition rounds, and rounding errors accumulate.
- **Whole rupees are exact:** 10.0, 12.0, 24.0 are exactly representable (below 2^53), and their sums are exact, so the error is 0.
- **Fixed width:** `int` has 32 bits, so a result above 2,147,483,647 wraps to a different number.
- **Division by 100:** `f / 100.0` introduces the fraction; `f` kept as paise never does.

10.6 Timing measurements from the benchmark program

`bench/bench.cpp` (compiled with `-O2`) repeats each operation 300 million times so that the cost per operation can be seen. The accumulator is `volatile` so the compiler cannot delete the loop.

Output of bench/bench.cpp

```
300M lookups: switch 0.279276 s, array 0.267015 s
300M peak checks: expression 0.38624 s, table 0.319004 s
read 3-value config file: 1.70027 microseconds per start-up
checksum 1200125001000
```

Comparison	Variant A	Variant B	Ratio
Fare lookup (300M)	`switch`: 0.279 s	array: 0.267 s	1.05 : 1
Peak check (300M)	expression: 0.386 s	table: 0.319 s	1.21 : 1
Addition (200M, Task 10 run)	`long long`: 0.07 s	`double`: 0.59 s	about 1 : 8
Fare table start-up	compiled-in: 0	config file: 1.7 microseconds	negligible

Threats to validity: one machine, one compiler (g++ 13.3.0), tight loops that stay in the CPU cache, and timings that vary a little between runs (a second run of the `long long` loop took 0.20 s instead of 0.07 s). The ratios are the evidence, not the exact seconds. Even the slowest figure is about 3 ns per addition, so 3,000,000 journeys cost a few milliseconds of arithmetic; the choice of type is therefore decided by **exactness**, not speed.

Alternative Concept and Complexity Analysis

Alternative: accumulate the 3,000,000-journey network total in `int`, `long long` or `double`.

Accumulator	What happens at 3,000,000 journeys (paise)	Failure type	Time (200M adds)	Space
`int`	6,000,000,000 > 2,147,483,647: result wraps (1,705,032,704)	**Overflow** (silent; undefined for signed)	about as fast as `long long`	4 bytes
`long long`	6,000,000,000 is exact; limit 9.22e18 (about 1.5e9 days of such traffic)	None in practice	0.07 s	8 bytes
`double` (rupees)	Whole-rupee fares exact; fractional fares carry error 4.4e-02 paise	**Loss of precision** (exact only up to 2^53 = 9.0e15 for integers)	0.59 s	8 bytes

All three are O(1) per addition and O(1) in space. `int` fails by **overflow** (a wrong number), `double` fails by **precision loss** (a slightly wrong number), and `long long` fails neither way at this scale. The two timings were measured in the same program (volatile accumulators, 200 million additions); the ratio matters more than the seconds.

Conclusion: `long long` paise is the only choice that is both exact and large enough.

TASK 11 - Recommend and Defend the Final Design

Bloom's Level: K5 - Evaluate | Marks: 4

11.1 Recommended design

Element	Recommendation	Evidence from my experiments
Money type	`long long` paise everywhere; `double` only for display	Task 3: `0.1` x 10 != 1.0; Task 10: drift 4.4e-02 paise vs 0 for paise
Day totals	`long long`	Task 10: 6,000,000,000 overflows `int` (wrapped to 1,705,032,704)
Constants	`constexpr` table and `enum class Zone`	Task 2: macro bug `DOUBLE_IT(3+1)` gave 5; `enum class` needs a cast
Peak rule	`isPeakHour()` with half-open windows	Task 6/9: 10:00 and 20:00 correctly off-peak, 8:00 and 17:00 peak
Fare lookup	`switch` for 3 zones; array (Task 12) for more	Task 6: equal speed (0.279 s vs 0.267 s)
Card	`SmartCard` class with a private balance	Task 7: public/global balance accepted -999; private rejected it
Input	`getline` + `stoi` with `try/catch`	Task 8: `2x` broke `cin >>` but was rejected by `stoi`
Conversions and warnings	Brace initialisation, `static_cast`, `-Wall -Wextra`	Task 4: `s < u` warning matched the wrong result
Tests	Table of cases run in a loop	Task 9: 19 of 19 pass, new case = one row

11.2 How a fare revision is carried out

Step	Action
1	Edit **one** line: the value in the `constexpr` fare table (for example `BASE_PAISE[2] = 3500`).

Step	Action
2	Add or change the matching rows in the test table (expected fares at 08:00, 10:00, 17:00, 20:00 for the changed zone).
3	Compile with `-Wall -Wextra`; the compiler reports any switch that misses a new enumerator.
4	Run the tests; all rows must pass before deployment.
5	Deploy the new executable to the gates and commit the change with a message that names the fare revision.

11.3 Limitations of the design

- **A revision needs recompilation and redeployment** to every gate. The configuration-file alternative below removes this at the cost of new failure modes.
- **Integer truncation:** `fare * 20 / 100` drops fractions of a paisa. For the current fares (multiples of 100 paise) the result is exact; a future fare such as 1,333 paise would lose 0.6 paise per journey. The rounding rule must be stated if that happens.
- **Concession rate is assumed** (50%). The assignment does not give it; the constant must be confirmed.
- **Single-threaded:** the program processes one journey at a time. Real gates run concurrently, so the card balance would need a lock or an atomic operation. This was not designed or tested here.
- **No persistence:** the balance lives in memory. A real system needs a database or the card chip, and a failure between `canTravel()` and `deduct()` must not lose money.
- **Time comes from the caller:** the code trusts `hour`. Clock errors at the gate would price journeys wrongly.

11.4 Does the design remain correct for the whole network?

Check	Calculation / measurement	Verdict
Largest day total (3,000,000 journeys, all at Rs 36 peak Outer)	3,000,000 x 3,600 = 10,800,000,000 paise (about 5x `INT_MAX`)	Needs `long long` - provided
Headroom of `long long`	9.22e18 / 1.08e10 = about 8.5e8 days	Safe

Check	Calculation / measurement	Verdict
Time per journey	O(1); 300 million lookups took 0.27 s, so about 1 ns each	3,000,000 journeys need well under a second of fare computation
Memory per card	8 bytes	10 million cards = 80 MB
Correctness of totals	Paise totals exact in all runs; double drifted	Correct

The fare logic therefore remains correct and fast at network scale. What the experiments do **not** show is behaviour under concurrency, disk or network delay, which is outside this assignment. The recommendation is limited to the fare logic itself.

11.4a The recommended code skeleton

The final design in one view. Every rule appears exactly once, and the full tested versions are in `Task09/task09.cpp` and `Task12/task12.cpp`.

Final design (outline)

```
// ---- constants: the only place fares and limits are written ----
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[]          = {1000, 2000, 3000};
constexpr int       PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE      = 5000;

// ---- rules: pure functions, easy to test ----
constexpr bool isPeakHour(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }
long long fareFor(Zone z, int hour);          // table lookup + integer surcharge, -1 if invalid
```

```
// ---- state: the balance is private, three guarded entry
points ----
class SmartCard {
public:
    bool canTravel(long long fare) const;      // fare >= 0 &&
balance_ - fare >= MIN_BALANCE_PAISE
    bool deduct(long long fare);             // changes nothing
if canTravel() is false
    bool recharge(long long paise);           // rejects paise
<= 0
private:
    long long balance_;
};
```

11.5 Summary of every alternative analysed and the decision

Task	Alternative studied	Result of the analysis	Decision
1	Procedural style	Same O(1) time/space; many more edits per rule change	Object-oriented
2	`#define` macros	Same speed; no type checking (macro bug gave 5)	`constexpr` + `enum class`
3	`double` rupees	Same speed and 8 bytes; error grows to 1.96e-06 Rs	`long long` paise
4	Implicit conversions	No run-time cost; more bugs found at compile time with casts	Explicit `static_cast` and `{}`
5	Peak lookup table	1.2x faster; 24 bytes; bounds risk	Expression
6	Array instead of `switch`	Equal speed; array scales better	`switch` now, array in Task 12
7	Public balance	Same cost; three rules breakable	Private balance
8	`getline` + `stoi`	Both O(L); rejects `2x`	`getline` + `stoi`

Task	Alternative studied	Result of the analysis	Decision
9	Hand-written tests	Both O(t); table adds a case in one row	Table of cases
10	`int` / `double` accumulators	`int` overflows; `double` loses precision	`long long`
11	Configuration file	1.7 microseconds start-up; run-time risk	Compiled-in for core rules

Every decision follows the same pattern: the alternatives usually have the **same complexity** (O(1) per journey, O(n) per day), so the choice is made on correctness and maintainability, and the evidence for each choice is a measured result from this report.

Alternative Concept and Complexity Analysis

Alternative: read the fare table from a configuration file when the gate starts.

Alternative: fares from fares.cfg

```
#include <fstream>
bool loadFares(const char* path, long long (&fare)[3])
{
    std::ifstream in(path);
    return  static_cast<bool>(in  >>  fare[0]  >>  fare[1]  >>
fare[2])      // all three read
           && fare[0] > 0 && fare[1] > fare[0] && fare[2] >
fare[1];    // sanity check
}
// file fares.cfg contains:  1000 2000 3000
```

Measure	Compiled-in `constexpr`	Configuration file
Start-up time for z fares	0 (values are in the executable)	O(z): measured **1.7 microseconds** to open and read 3 values (file cached by the OS)

Measure	Compiled-in `constexpr`	Configuration file
Space for z fares	z x 8 bytes in the executable	z x 8 bytes in memory (24 bytes), plus the file on disk
Cost of a fare revision	Edit, recompile, test, redeploy executable to every gate	Edit one text file and restart or reload
Type safety	Compiler checks types and bounds	Run-time checks only (missing file, letters, negative values)
Failure modes	Cannot start with a wrong table	Missing, corrupt or hand-edited file; needs defaults and validation
Audit trail	Version control of the source	Needs separate control of the file

The extra start-up time (1.7 microseconds for 3 values, O(z)) and 24 bytes are negligible; the real trade-off is **safety versus agility**. **Recommendation:** keep the compiled-in table for the safety-critical rules (zones, minimum balance), and consider a validated configuration file for the fare amounts alone if revisions are frequent.

TASK 12 - Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Bloom's Level: K5 - Evaluate | Marks: 8

Changed requirement: a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass with unlimited journeys in two chosen zones.

Assumptions: *on a weekend the 10% discount applies and the peak surcharge is* ***not*** *charged (the two never combine); on a weekday only the surcharge can apply. A pass covers journeys that end in either of its two zones (any two of the four, including Airport) and charges Rs 0. Pass expiry dates are not modelled.*

A. Impact Analysis (2 marks)

Part of the original design	Affected ?	What must change
`enum class Zone`	Yes	Add `Airport = 3`
`BASE_PAISE[]` constant	Yes	Fourth entry 6000 (Rs 60)
New constants	Yes	`WEEKEND_DISCOUNT_PERCENT = 10`, `ZONE_COUNT = 4`
`fare(Zone, bool peak)` signature	Yes	Needs a `weekend` input; a `bool peak` is no longer enough
`switch` on the zone	Yes	New `case Airport`; the old `default` would silently charge Rs 0
Peak/surcharge expression	Yes	Must be bypassed on weekends (no combination)
`isPeakHour()`	No	Unchanged
`canTravel()` / minimum balance rule	No	Unchanged (still Rs 50)
`SmartCard` class	Yes	Must know its pass and skip the fare for covered zones
New class	Yes	`MonthlyPass` (two zones, `covers()`)

Part of the original design	Affected ?	What must change
Test table	Yes	New rows for Airport, weekend, pass

B. Evaluation of the Original Design (2 marks)

Worked example 1 - Airport fare. The original `fare()` switches on three zones. An Airport journey (value 3 in the new enum) reaches the `default` branch and the fare is computed as **0**: the airport is free, with no error. If the fare is read as `BASE_PAISE[3]` instead, the access is outside the 3-element array (undefined behaviour). The switch cannot absorb the new zone without a code change in two places.

Worked example 2 - weekend discount. Original: `fare = base; if (peak) fare += fare * 20 / 100;`. On a Saturday at 9 AM in Middle zone the original gives 2000 + 400 = **2400**, but the new rule requires 2000 - 200 = **1800**. A 10% discount cannot be bolted on after the surcharge, because the two rules must not combine: applying a discount after the surcharge would give 2160 (also wrong). The function needs a different structure.

Worked example 3 - monthly pass. `SmartCard::deduct(fare)` knows only a balance. It cannot know that a Middle-zone journey is free for a pass holder. The class can be kept, but it needs a pass member and a new entry point (`travel(zone, hour, weekend)`).

Component	Absorbs the change?	Verdict
Constants, `enum class`, `long long` paise, `isPeakHour`, `canTravel`	Yes	Retain
`SmartCard` (private balance, `deduct`, checks)	Mostly: needs a pass and a new method	Modify
`switch`-based `fare(zone, peak)`	No: wrong signature, wrong default, no weekend	Redesign this function

C. Adaptation: the revised design (2 marks)

Decision: MODIFY. Keep the typed constants, integer paise, `enum class`, the private balance and the Rs 50 rule. Replace the `switch` with a fare table indexed by zone, move the weekend/peak decision into one separate rule function, and add a `MonthlyPass` class.

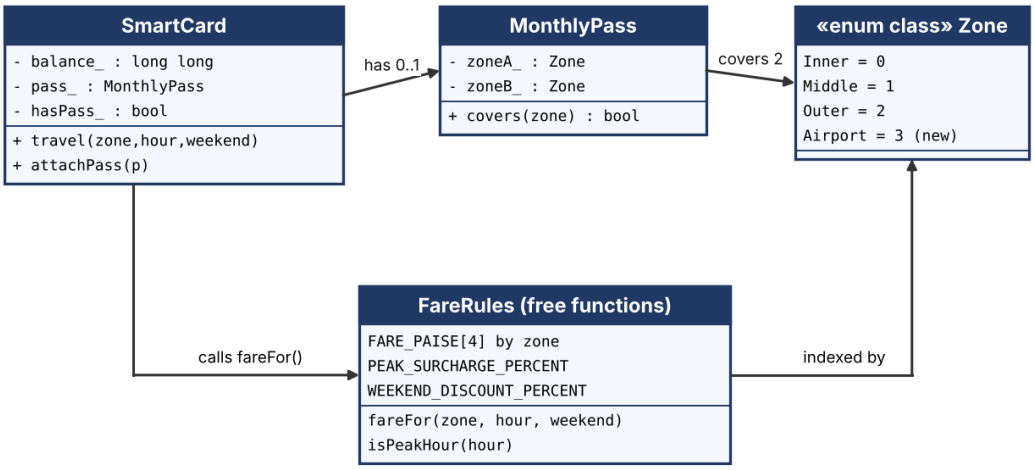


Figure 12.1 - UML class diagram of the revised design

Source: Task12/task12.cpp (revised design)

```
// Task 12 - redesigned fare system: 4 zones, weekend
discount, monthly pass
#include <iomanip>
#include <iostream>
#include <string>

enum class Zone { Inner = 0, Middle = 1, Outer = 2, Airport =
3 };
constexpr int ZONE_COUNT = 4;

constexpr long long FARE_PAISE[ZONE_COUNT] = {1000, 2000,
3000, 6000}; // one place to edit
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

```
constexpr bool isPeakHour(int h) { return (h >= 8 && h < 10)
|| (h >= 17 && h < 20); }

// Assumption: the weekend discount and the peak surcharge
never combine. On a weekend the
// 10% discount applies and the surcharge is NOT charged; on a
weekday only the surcharge can apply.
long long fareFor(Zone z, int hour, bool weekend)
{
long long base = FARE_PAISE[static_cast<int>(z)];
if (weekend) return base - base * WEEKEND_DISCOUNT_PERCENT /
100;
return isPeakHour(hour) ? base + base * PEAK_SURCHARGE_PERCENT
/ 100 : base;
}

class MonthlyPass {
public:
MonthlyPass(Zone a, Zone b) : a_{a}, b_{b} {}
bool covers(Zone z) const { return z == a_ || z == b_; }
private:
Zone a_, b_;
};

class SmartCard {
public:
explicit SmartCard(long long paise) : balance_{paise} {}
void attachPass(const MonthlyPass& p) { pass_ = p; hasPass_ =
true; }

// returns the amount charged, or -1 if journey is denied
long long travel(Zone z, int hour, bool weekend)
{
if (hasPass_ && pass_.covers(z)) return 0;
```

```
long long f = fareFor(z, hour, weekend);
if (balance_ - f < MIN_BALANCE_PAISE) return -1;
balance_ -= f;
return f;
}
long long balance() const { return balance_; }
private:
long long balance_;
bool hasPass_{false};
MonthlyPass pass_{Zone::Inner, Zone::Inner};
};

int main()
{
struct T { const char* name; Zone z; int h; bool we; long long
expected; };
T tests[] = {
{"Airport weekday off-peak", Zone::Airport, 12, false, 6000},
{"Airport weekday peak",     Zone::Airport,  9, false, 7200},
{"Middle weekend off-peak",  Zone::Middle,  12, true,  1800},
{"Middle weekend peak",      Zone::Middle,   9, true,  1800},
{"Outer weekday peak",       Zone::Outer,   18, false, 3600},
```

Output of Task12/task12.cpp

```
Case                        Expected  Actual    Result
Airport weekday off-peak    6000      6000      PASS
Airport weekday peak        7200      7200      PASS
Middle weekend off-peak     1800      1800      PASS
Middle weekend peak         1800      1800      PASS
Outer weekday peak          3600      3600      PASS
Inner weekend off-peak      900       900       PASS

--- monthly pass for Inner + Middle ---
```

```
Middle peak (covered) charged 0, balance 20000
Airport #1 charged 6000, balance 14000
Airport #2 charged 6000, balance 8000
Airport #3 charged -1 (-1 = denied), balance 8000

Passed 10 of 10 checks
```

D. Evidence-Based Defence (2 marks)

Test	Rule checked	Expected	Actual	Result
Airport weekday off-peak	New zone fare	6000	6000	PASS
Airport weekday peak (09:00)	Surcharge on new zone	7200	7200	PASS
Middle weekend off-peak	10% discount	1800	1800	PASS
Middle weekend peak (09:00)	No combination of discount and surcharge	1800	1800	PASS
Outer weekday peak (18:00)	Old rule unchanged	3600	3600	PASS
Inner weekend 10:00	Discount: 1000 - 100	900	900	PASS
Pass: Middle zone (covered)	Free journey	0	0	PASS
Airport x 2 with Rs 200 card	Not covered: charged 6000 each	6000, 6000	6000, 6000	PASS
Airport 3rd time (balance 8000)	Rs 50 minimum: 8000 - 6000 = 2000 < 5000	-1 (denied)	-1	PASS
Total	10 checks	10 of 10	10 of 10	**PASS**

Why modify rather than redesign everything. The measured evidence supports keeping the foundations: integer paise stayed exact (all fares are whole-paise results of integer arithmetic), the Rs 50 rule needed no change because it lives in one place, and the old Outer weekday-peak case still gives 3600. Only the part that failed the worked examples (the

`switch`-based `fare`) was replaced. Redesigning the whole system would have discarded tested code for no gain.

Trade-off	Table-driven fare + rule function + Pass class	Original switch design
Adding a fifth zone	One enumerator, one table value, `ZONE_COUNT` + 1	New `case`, new array value, signature review
Speed	O(1) (the array was as fast as the switch: 0.267 s vs 0.279 s per 300M lookups)	O(1)
Unknown zone	Needs an explicit range check (a limitation of tables)	`default` branch
Weekend / peak interaction	Visible in one function (`fareFor`)	Scattered across `if` statements
Memory per card	+ pass fields (two zones and a flag: about 12 bytes, O(1))	8 bytes

Limitations and risks: pass validity period (a monthly pass must expire), refunds, and the exact meaning of 'does not combine' were not specified, so the assumption stated above must be confirmed. The unit tests cover the rules above, not concurrency or card hardware.